

## Determine a Ordem

- **Limite de tempo:** 10 minutos
- **Ambiente:** uma GPU ( $\approx 16$  GB VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb`  $\leq 1$  MB
- **Armazenamento:** 5 GB

### Problema

São-lhe fornecidos diálogos falados em inglês entre dois participantes, o *Orador A* e o *Orador B*. Cada diálogo está segmentado em turnos de fala, contendo cada turno a fala de apenas um orador. Cada turno está armazenado como um ficheiro de áudio `.wav` separado, sendo que um diálogo completo é representado por um conjunto de ficheiros `.wav`, um por cada turno.

Infelizmente, os turnos foram baralhados aleatoriamente, resultando em que a conversa deixe de fazer sentido. No nome de ficheiro `chunk_{k}.wav`,  $k$  refere-se ao  $k$ -ésimo segmento no conjunto baralhado, e não ao  $k$ -ésimo turno no diálogo original.

**!! A sua tarefa é reconstruir a ordem cronológica original da conversa.**



## Dataset

Cada diálogo contém  $n$  ficheiros de áudio denominados por: `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Os segmentos são turnos individuais. Os nomes dos ficheiros correspondem apenas à ordem baralhada. Não indicam a posição de um segmento na conversa original. Cada diálogo tem 7–20 segmentos, mono, 44.1 kHz (pode fazer resampling).

**`prefix.json` contém os índices dos nomes de ficheiro dos dois primeiros segmentos para cada diálogo.** Isto identifica o verdadeiro início do diálogo e elimina a ambiguidade entre ler a conversa para a frente ou para trás.

Por exemplo: 11: [7, 12] significa que o primeiro e o segundo turno do diálogo 11 são `chunk_7.wav` e `chunk_12.wav`, respetivamente.

### O que recebe

Recebe **duas pastas com formato idêntico:**

| Pasta                             | Diálogos | <code>answers.json</code> ? | Utilize-a para                                 |
|-----------------------------------|----------|-----------------------------|------------------------------------------------|
| <code>dataset/train/</code>       | 1,288    | ☑ incluído                  | treinar / fazer fine-tuning do modelo          |
| <code>dataset/test_public/</code> | 100      | ☑ incluído                  | executar a pipeline e autoavaliá-lo localmente |

Durante a avaliação, a sua pasta `dataset/test_public/` vai ser substituída de forma transparente por uma `hidden evaluation set` (`test_leaderboard_a` para a tabela classificativa pública e `test_leaderboard_b` para a tabela classificativa final) — estas têm o mesmo tamanho e formato que `dataset/test_public/`, mas sem `answers.json`.

O seu notebook é executado novamente sobre esses dados, e o ficheiro `answers.json` que produz é utilizado para a pontuação. Os diálogos de teste reservados provêm da mesma distribuição que `train`, pelo que a sua pontuação `test_public` local constitui uma previsão fiel.

## Estrutura de diretórios

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

## Saída

Para cada diálogo, determine a ordem cronológica original dos respetivos segmentos de áudio. A sua previsão deve ser uma permutação  $P$  de  $\{0, 1, \dots, n-1\}$ , em que  $P[i]$  é a posição cronológica prevista de `chunk_i.wav` (0 = primeiro).

O seu ficheiro de saída `answers.json` deve associar cada ID de um diálogo à respetiva permutação prevista:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

## Exemplo

Um diálogo tem 3 segmentos baralhados `chunk_0`, `chunk_1`, `chunk_2`:

| segmento<br>baralhado | conteúdo falado                                     | posição verdadeira<br>(ordem) |
|-----------------------|-----------------------------------------------------|-------------------------------|
| chunk_0.wav           | "No worries — I'll send you the notes afterwards."  | 2 (último)                    |
| chunk_1.wav           | "Hey, are you coming to the three o'clock meeting?" | 0 (primeiro)                  |
| chunk_2.wav           | "I can't — I've got a dentist appointment then."    | 1                             |

A ordem verdadeira é **chunk\_1** → **chunk\_2** → **chunk\_0**, pelo que  $P = [2, 0, 1]$ , e `prefix.json` contém  $[1, 2]$ .

⚠ **P deve ser uma permutação genuína:** comprimento  $n$ , indexada a partir de 0, com cada valor exatamente uma vez. Duplicados, valores em falta ou entradas fora do intervalo (por exemplo, indexadas a partir de 1) resultam numa pontuação de 0 para esse diálogo, tal como um diálogo ausente do ficheiro. Um ficheiro malformatado ou que não esteja em JSON é rejeitado.

## Pontuação

A pontuação desta tarefa é a **taxa de acerto da ordenação por pares**. São verificados todos os pares de segmentos aos quais é colocada a questão: *qual dos dois deve aparecer primeiro?* Um par está correto se a sua previsão der a mesma resposta que a verdade de referência. Para um diálogo com  $n$  segmentos, existem

$$M = n(n - 1)/2$$

pares; seja  $I$  o número de inversões — pares ordenados de forma diferente da verdade de referência:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

! **A pontuação final é a média das pontuações por diálogo sobre todos os diálogos.**

## Modelos permitidos

Só pode utilizar os seguintes modelos pré-treinados para resolver esta tarefa, tanto durante o treino como durante a avaliação. Todos estes modelos já estão descarregados e disponíveis no ambiente. Pode consultar exemplos da sua utilização no notebook de baseline `solution.ipynb`. Note que não pode utilizar qualquer outro modelo e que o seu programa não tem acesso à internet.

- **Representações de fala: wav2vec 2.0.** O **encoder do Whisper** também pode ser utilizado como extrator de características. [Informação do modelo wav2vec](#)
- **Reconhecimento automático de fala (ASR): OpenAI Whisper** (qualquer tamanho). [Informação do modelo Whisper](#)
- **Modelo de linguagem: Qwen2.5-0.5B**, que pode ser utilizado em modo zero-shot ou submetido a fine-tuning na partição `train` fornecida. [Informação do modelo Qwen](#) Note que o limite de 10 minutos deve abranger qualquer treino ou fine-tuning que realize durante a avaliação, além da inferência no conjunto de avaliação.

## Como submeter

- Abra `solution.ipynb` e execute todas as células. Confirme que este escreve `answers.json` no diretório de trabalho com uma permutação para cada diálogo em `dataset/test_public/` (100 diálogos). Durante a avaliação, o notebook é novamente executado no conjunto de teste oculto, e o `answers.json` aí produzido vai ser usado para calcular a pontuação.
- Melhore a solução, se quiser — ou não; a baseline, por si só, valida a pipeline.
- Abra o separador Git na barra lateral esquerda do JupyterLab.
- Coloque `solution.ipynb` em **Stage** (o símbolo + junto ao mesmo).
- Introduza uma mensagem de commit e clique em **Commit**.
- Clique no ícone da nuvem com uma seta para cima para fazer push.
- Regresse a esta página do concurso e clique em **Submit**.

Submeta exatamente um ficheiro, denominado `solution.ipynb`.